

BONUS LECTURE #2

Skills 2.0 작동 로직 + Sub-agent 생성·협업

| description · Progressive Disclosure · 컨텍스트 격리
본편을 완성하는 마지막 조각

SDD 보너스 #2 (15~18분)

By FlowCoder (박현일)

왜 이 강의가 필요한가

보너스 1 복습

커맨드 → 스킬 마이그레이션

- ✓ 경로 · frontmatter만 바꾸면 끝
- ✓ "어떻게 옮기는가"에 집중

보너스 2 예고

Skills 2.0 내부 원리 + Sub-agent 협업 패턴

- ✓ Claude는 어떻게 스킬을 고르나?
- ✓ Skill과 Sub-agent는 뭐가 다른가?

원리를 알아야 새 스킬·서브에이전트를 직접 설계할 수 있습니다.

스킬 호출 3단계 — Scan · Match · Load

01 Scan — frontmatter만 훑는다

`.claude/skills/*/SKILL.md` 의 **description·name**만 먼저 읽음. 본문은 아직 안 읽음.

02 Match — 의도와 description 비교

사용자 요청과 각 스킬의 description을 비교해 가장 관련 있는 스킬 선정.

03 Load — 본문과 references 로드

선정된 스킬의 `SKILL.md` 본문을 읽고, 필요시 `references/` · `scripts/` 도 불러옴.

핵심 비유: description = 목록 페이지의 제목, SKILL.md 본문 = 클릭해서 보는 상세 페이지. 제목 품질이 스킬 품질을 결정한다.

👉 description 3요소 — WHAT · WHEN · WHO

WHAT

무엇을 하는지

"SDD 방법론으로 PRD를 작성"

WHEN

언제 호출되는지

"'기획', 'PRD', 'requirements' 요청 시"

WHO

누가 쓰는지 (선택)

"기획자·개발자가 사용"

❌ 나쁜 예

```
description: 프로젝트 관리용 스킴
```

WHAT만 있고 WHEN 없음 → 언제 호출할지 Claude가 판단 못 함

✅ 좋은 예

```
description: SDD 방법론으로  
PRD를 작성할 때 사용.  
사용자가 "기획", "PRD",  
"요구사항 정리"를 요청하거나  
"/sdd:plan"을 입력하면 자동 호출.
```

Progressive Disclosure — 왜 폴더 구조인가

스킬 폴더 구조

```
skills/  
└─ sdd-plan/  
   │ └─ SKILL.md           ← 메인 (항상 로드)  
   │ └─ references/  
   │     └─ templates.md ← 필요할 때만  
   └─ scripts/  
       └─ helper.sh       ← 필요할 때만
```

책 비유

- › **SKILL.md** = 목차 · 항상 펼침
- › **references/** = 부록 · 필요시만
- › **scripts/** = 도구 · 실행 가능

Claude는 필요할 때만 부록을 펼침 → 토큰 절약

설계 팁: SKILL.md는 100~300줄로 핵심만. 길어지면 references로 분리. 책 목차가 500페이지면 이상하다.

⚡ 토큰 효율성 — 커맨드보다 스킬이 이기는 이유

단계	커맨드 6개	스킬 6개
시작 시	6개 파일 본문 전부 시스템 프롬프트에 로드	6개 <code>description</code> 만 로드
사용자 요청 도착	전부 이미 메모리에 있음	매칭된 스킬의 <code>SKILL.md</code> 만 추가 로드
상세 템플릿 필요 시	이미 로드됨 (쓰든 안 쓰든)	<code>references/</code> 를 그때만 펼침
예상 토큰 (개념적)	~10,000+ (본문 전체)	~600 (description만)

자동 호출은 편의, 토큰 효율성은 성능.
긴 대화·큰 파일·복잡한 디버깅에서 차이가 극적으로 납니다.

Sub-agent란 무엇인가

독립된 컨텍스트

메인 대화와 격리된 **별도 세션**. 메인의 잡음이 따라가지 않음.

전용 시스템 프롬프트

"당신은 13년차 기획자" 같은 역할이 **사전 고정**. 매번 지정할 필요 없음.

제한된 권한

`tools`, `model` 을 frontmatter에서 지정. **최소 권한** 원칙 적용.

한 방향 통신

부모 → 서브에이전트만 가능. 서브는 **최종 결과만 반환**. 중간 질문 불가.

비유: Skill은 "사전에 정의된 레시피", Sub-agent는 "전용 셰프 (주방을 따로 씀)".

Skill vs Sub-agent

항목	Skill	Sub-agent
용도	재사용 가능한 지식·프로세스	독립된 작업 위임
컨텍스트	공유 (부모 대화에 주입)	격리 (별도 공간)
호출	description 기반 자동 호출	Task/Agent 도구로 명시 호출
비용	낮음 (메인 모델 재활용)	높음 (별도 모델 호출)
반환	본문 가이드에 따라 자유	최종 요약만

기억할 단 한 줄 — "공유냐, 격리냐."

Sub-agent 생성 구조

경로

```
.claude/
└─ agents/
    └─ planner.md ← 파일 하나
```

스킬과 달리 **폴더가 아니라 파일 하나**. references/scripts 구조 없음.

PLANNER.MD 내부

```
---
name: planner
description: PRD 작성 전문가.
             sdd-plan 스킬이 위임할 때 호출.
tools: Read, Write, Grep
model: sonnet
---

# 역할
당신은 13년 경력의 기획자입니다.

# 작업 절차
1. 요구사항 파악
2. 사용자 스토리 작성
3. 기능 명세 정의
...
```

frontmatter 심화 — tools · model 선택 전략

TOOLS — 최소 권한 원칙

- > `@planner` → `Write` (PRD만 작성)
- > `@reviewer` → `Read, Grep` (읽고 피드백)
- > `@developer` → `Read, Write, Edit, Bash`

권한을 줄일수록 에이전트가 집중

MODEL — 작업 성격에 맞추기

Haiku	탐색·요약·분류
Sonnet	구현·리뷰 (기본)
Opus	아키텍처·전략

항상 최고 모델 = 초보의 실수

SDD 실전 예: `@planner=sonnet · @architect=opus · @developer=sonnet · @reviewer=sonnet`

Skill이 Sub-agent를 호출하는 흐름

- 01 사용자: "할일 앱 PRD 써줘"
- 02 Claude 메인 → `sdd-plan` 스킬 자동 호출 (description 매칭)
- 03 `sdd-plan` 의 SKILL.md 본문 실행 — "복잡한 기획은 @planner에 위임" 가이드
- 04 Claude → Task 도구로 `@planner` 호출 → 컨텍스트 격리 시작
- 05 `@planner` 격리 공간에서 PRD.md 생성 (기획자 역할로 집중)
- 06 PRD.md 경로 + 요약만 메인 대화로 반환

SDD 팀 구성도 — 진입(Skill) → Agent → 내부 Skill

진입 Skill (자연어 또는 슬래시)	Agent (격리 작업)	내부 Skill (지식·템플릿)
<code>/sdd-toolkit:init</code> · "워크스페이스 셋업"	(직접 처리)	—
<code>/sdd-toolkit:plan</code> · "PRD 써줘"	<code>@planner</code>	<code>prd-writer</code>
<code>/sdd-toolkit:design</code> · "기술 설계해줘"	<code>@architect</code>	<code>tech-spec-writer</code>
<code>/sdd-toolkit:build</code> · "코드 짜줘"	<code>@developer</code>	<code>code-generator</code>
<code>/sdd-toolkit:review</code> · "검증해줘"	<code>@reviewer</code>	<code>spec-validator</code>

① 진입 Skill

사용자 자연어를 받아 description 매칭으로 자동 호출. SDD 워크플로우의 시작.

② Agent (격리)

전용 페르소나로 별도 컨텍스트에서 작업. 메인 대화의 잡음 차단.

③ 내부 Skill

Agent가 활용하는 지식·템플릿. 본편의 4개 스킬이 그대로.

3-레이어: Agent의 frontmatter `skills: [prd-writer]` 로 내부 스킬 매핑 — 본편이 이미 이 구조로 설계됨

Plan 흐름 실전 추적

사용자 입력: "할일 앱 PRD 써줘"

Step	레이어	동작
1	Claude 메인	"PRD" 키워드 감지 → description 매칭
2	Skill	<code>sdd-plan</code> 자동 호출 → SKILL.md 로드
3	Claude 메인	요구 복잡도 판단 → 위임 결정
4	Task 도구	<code>@planner</code> 호출 (tools: Write, model: sonnet)
5	Sub-agent	격리 공간에서 PRD.md 생성 (기획자 역할)
6	Claude 메인	PRD 경로 + 요약 반환 → 사용자에게 전달

전체 소요 ~1분. Skill이 프로세스, Sub-agent가 작업.

3가지 협업 패턴

① Skill-only

가장 가벼움. 스킬 하나로 끝.

- 예: `flowcoder-brand-kit`
 - 브랜드 정보 주입만

② Skill → Sub-agent

본편 SDD 기본 패턴.

- 예: `sdd-plan` → `@planner`
 - 무거운 작업 위임

③ Multi-agent 병렬

독립 작업 다수 동시 수행.

- 예: 경쟁사 5개 동시 리서치
 - Fan-out/Fan-in 패턴

Academy 강의 5 "프로젝트 통째로 시키기"에서 패턴 5개를 더 다룹니다.

언제 Skill? 언제 Sub-agent?

상황	선택
재사용 가능한 지식·템플릿 주입	Skill
큰 작업 — 컨텍스트가 지저분해질 때	Sub-agent
같은 프롬프트를 여러 번 쓸 때	Skill
여러 작업 병렬 수행	Multi Sub-agent
도구 권한을 제한하고 싶을 때	Sub-agent
공식 프로세스를 강제하고 싶을 때	Skill

한 줄 요약 — "공유냐, 격리냐."

✓ 핵심 정리

1. 스킬은 **Scan → Match → Load** 3단계 — description이 목차, 본문이 상세
 2. **Progressive Disclosure**로 토큰 효율성 확보 — SKILL.md + references/ + scripts/
 3. Sub-agent는 **컨텍스트 격리 + 전용 프롬프트 + 권한 제한** — `.claude/agents/*.md`
 4. Skill과 Sub-agent는 대체재 아닌 **보완재** — "공유냐 격리냐"로 결정
-

원리를 알면, 직접 설계할 수 있다.

본편 + 보너스 1 + 보너스 2

Skills 2.0 시대의
완전한 SDD 시스템

본편 — 구조와 전체 그림

보너스 1 — 최신 Claude Code 업그레이드

보너스 2 — 원리와 협업 패턴

FlowCoder 박현일

질문은 강의 커뮤니티에서